

```

def rollout(policy, dataset, number_sample, max_steps):
    rollout_dataset = []
    for batch in dataset:
        batch = batch.repeat(number_sample)
        # Initialize parallel environments
        envs = env_process_pool.submit(batch.initialize)
        states = env_process_pool.submit(envs.setup)
        for t in range(max_steps):
            actions = policy.generate(states)
            # Reuse final action estimates from denoising
            + action_estimates = policy.denoising_predictions
            + variance = denoising_variance(action_estimates)
            - rollout_dataset.append((envs, states, actions))
            + rollout_dataset.append((envs, states, actions, variance))
            states, dones = env_process_pool.submit(envs.step, actions)
            # Remove completed tasks
            active = [(e,s) for e,s,d in zip(envs,states,dones) if not d]
            if not active:
                break
            envs, states = zip(*active)
        return rollout_dataset

    # Collect rollouts and update the policy
    rollout_dataset = rollout(policy, dataset, number_sample, max_steps)
    rewards = outcome_rewards(rollout_dataset)
    advantages = group_advantages(rewards)
    + weights = two_level_weights(rollout_dataset)
    + advantages = reweight(advantages, weights)
    policy.update(rollout_dataset, advantages)

```